FL

คู่มืออธิบายการทำงานของโค้ดทั้งระบบ

Clover Project — ระบบบริหารจัดการสหกรณ์รับซื้อน้ำยางพารา (Field Latex)

เอกสารฉบับสมบูรณ์สำหรับนักพัฒนาและผู้เริ่มต้น อธิบายโครงสร้างสถาปัตยกรรม ฐานข้อมูล ระบบเซิร์ฟเวอร์ การเชื่อมโยง API หน้าบ้าน และตรรกะทางธุรกิจอย่างละเอียด

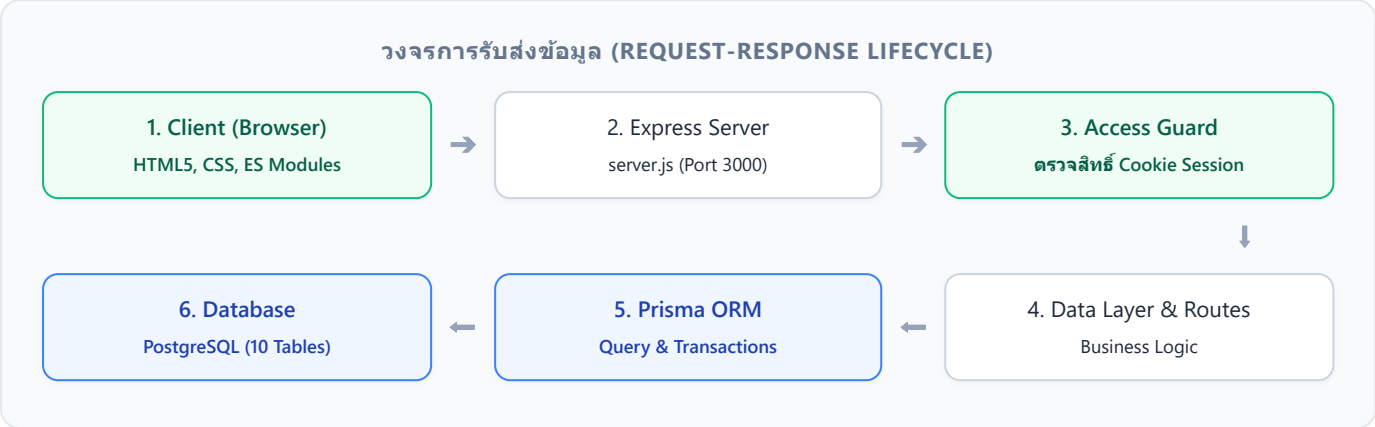
เทคโนโลยีหลัก: Node.js • Express • PostgreSQL • Prisma ORM • Vanilla HTML/CSS/JS
เวอร์ชันเอกสาร: 1.0 (อัปเดตระบบล็อกเงินปันผลและการเบิกเงิน)
วันที่จัดทำ: ตุลาคม 2569

สารบัญเนื้อหา

1. ภาพรวมสถาปัตยกรรมระบบ (Architecture)
2. ฐานข้อมูล 10 ตารางและการเชื่อมโยง (Database)
3. ระบบหลังบ้าน (Backend Server & Middleware)
4. ระบบความปลอดภัย & Dual-Session
5. ชั้นตรรกะข้อมูลและ API Endpoints (Data Layer)
6. ระบบหน้าบ้าน (HTML, CSS & DOM Engine)
7. ส่วนประกอบที่นำมาใช้ร่วมกัน (Shared Modules)
8. สูตรคำนวณและตรรกะทางธุรกิจที่สำคัญ

1. ภาพรวมสถาปัตยกรรมระบบ (System Architecture)

ระบบ **Clover Project** ถูกออกแบบขึ้นเพื่อบริหารงานสหกรณ์กองทุนสวนยาง ครอบคลุมงานรับซื้อน้ำยางสดประจำวัน, การแบ่งรายได้ระหว่างเจ้าของสวนกับลูกจ้างกรีดยาง, การเบิกเงินสะสม, การจ่ายเงินปันผลประจำปี และระบบพอร์ทัลสำหรับสมาชิก สถาปัตยกรรมของโปรเจกต์เป็นแบบ **Monolithic Architecture** ที่ทำงานบน Node.js + Express โดยแยกการทำงานออกเป็น 4 ชั้นหลักอย่างชัดเจน:



เลเยอร์	เทคโนโลยี / ไฟล์หลัก	หน้าที่ความรับผิดชอบ
Presentation Layer	`pages/*.html` , `public/js/pages/*` , `public/css/*`	แสดงผล UI, ผูกข้อมูลผ่าน Template Tag, รับ Action จากผู้ใช้ และติดต่อ API ด้วย Fetch
Routing & Guard	`server/routes/*` , `access-guard.js`	ตรวจสอบการยืนยันตัวตน, ดักสิทธิ์ ADMIN/STAFF/MEMBER และแปลง Request Body
Data Layer	`server/lib/data/*`	คำนวณสูตรธุรกิจ, ควบคุมความถูกต้อง (Validation), รัน Database Transactions
Database Layer	PostgreSQL via `@prisma/client`	จัดเก็บข้อมูลถาวร, ควบคุม Foreign Key Constraints, Unique Constraints

2. ฐานข้อมูล 10 ตารางและการเชื่อมโยง (Database & Relationships)

ฐานข้อมูลถูกนิยามในไฟล์ `prisma/schema.prisma` ประกอบด้วยทั้งหมด 10 ตาราง โดยมี **Member (สมาชิก)** เป็นหัวใจศูนย์กลางของระบบการเงิน:

ชื่อตาราง (Model)	บทบาทหน้าที่	คีย์หลัก (PK) & ข้อกำหนด (Constraints)	ความสัมพันธ์ (Relations)
Member	สมาชิกเจ้าของสวนยาง เก็บค่ายอดเงินกระเป๋าสะสม (`walletBalance`) และปันผลสะสม (`dividendBalance`)	PK id UQ memberCode, idCardNumber	1:N กับ MePair, Purchase, Withdrawal, DividendPayment
Employee	ลูกจ้างกรีดยางและส่งน้ำยางหน้างานแทนสมาชิก	PK id UQ employeeCode, idCardNumber	1:N กับ MePair, 0..1:N กับ Purchase
MePair	สัญญาจับคู่สมาชิก-ลูกจ้าง พร้อมสัดส่วนรายได้ (`memberShare` + `employeeShare` = 100)	PK id UQ pairCode	FK memberId ➜ Member FK employeeId ➜ Employee
Purchase	บันทึกการรับซื้อน้ำยางสดประจำวัน คำนวณน้ำหนักแห้ง ยอดรวม และส่วนแบ่งรายได้	PK id UQ purchaseCode	FK memberId ➜ Member FK employeeId ➜ Employee (nullable)
Withdrawal	บันทึกการเบิกเงินสดจากยอดสะสมของสมาชิก	PK id UQ withdrawalCode	FK memberId ➜ Member
DividendPayment	ประวัติการจ่ายเงินปันผลประจำปี พ.ศ. ให้สมาชิก (1 แถวต่อสมาชิกต่อปี)	PK id UQ dividendCode, `[memberId, buddhistYear]`	FK memberId ➜ Member
Staff	บัญชีเจ้าหน้าที่/ผู้ดูแลระบบ (Role: `STAFF` หรือ `ADMIN`)	PK id UQ staffCode, username, email	1:N กับ ActivityLog
ActivityLog	บันทึก Audit Log กิจกรรมการทำงานของเจ้าหน้าที่ (Login, Create, Update, Delete)	PK logId (BigInt)	FK staffId ➜ Staff
ReferencePrice	ราคากลางประจำวัน กำหนดโดย Admin (1 วันมี 1 ราคา)	PK id UQ date	Standalone (เชื่อมโยงตามวันที่)
ReferencePriceHistory	ประวัติการบันทึกหรือเปลี่ยนราคากลาง (Insert-only)	PK id	Standalone (เชื่อมโยงตามวันที่)

> **ข้อกำหนดความปลอดภัยของฐานข้อมูล (Database Constraints):**
> ตาราง `DividendPayment` มีการกำหนด `@@unique([memberId, buddhistYear])` ซึ่งรับประกันในระดับฐานข้อมูลว่าสมาชิก 1 คน จะไม่มีทางได้รับเงินปันผลซ้ำสองครั้งในรอบปี พ.ศ. เดียวกัน แม้จะมีการส่งคำขอพร้อมกัน (Concurrency) ก็ตาม

3. ระบบหลังบ้าน (Backend Server & Middleware)

เซิร์ฟเวอร์เริ่มต้นทำงานจากไฟล์ `server/server.js` ซึ่งมีลำดับการประมวลผลคำขอ (Middleware Pipeline) ดังนี้:

1. **โหลดค่าคอนฟิก (`server/env.js`):** ตรวจสอบไฟล์ `.env` และตั้งค่าโหมดการทำงาน `NODE_ENV`
2. **ให้บริการไฟล์คงที่ (Static Assets):** ให้บริการไฟล์จาก `public/` (CSS, JS, Fonts, Images) และ `shared/` ผ่าน `express.static`
3. **อ่านเนื้อหา Body:** เปิดรับ Body แบบข้อความดิบขนาดสูงสุด 50MB (`express.text({ limit: "50mb" })`) เพื่อรองรับการอัปโหลดรูปภาพแบบ Base64 และแปลงเป็น JSON ผ่าน `readJsonBody()`
4. **ด่านตรวจสิทธิ์ (`server/middleware/access-guard.js`):** ตรวจสอบ Cookie และสิทธิ์การเข้าถึงก่อนอนุญาตให้ผ่านไปยัง Route
5. **เราต์ API (`server/routes/api.routes.js`):** นำส่งคำขอไปยัง API Controller ต่าง ๆ ภายใต้พาธ `/api/*`
6. **เราต์หน้าเว็บ HTML (`server/routes/pages.routes.js`):** ส่งหน้าเว็บ HTML พร้อมแทรก Layout Data
7. **ด่านจับข้อผิดพลาดส่วนกลาง:** หน้า 404 Not Found และ Global 500 Error Handler

4. ระบบความปลอดภัย & Dual-Session (ระบบเซสชัน 2 ระดับ)

ระบบ Clover Project มีจุดเด่นด้านความปลอดภัย คือการแยก Cookie เซสชันและอัลกอริทึมการเข้ารหัสระหว่าง **เจ้าหน้าที่** และ **สมาชิก** ออกจากกันโดยสิ้นเชิง:

คุณสมบัติ	ฝั่งเจ้าหน้าที่ (Staff / Admin)	ฝั่งสมาชิก (Member Portal)
ชื่อ Cookie	`session`	`member_session`
ข้อมูลการยืนยันตัวตน	Username + Password (scrypt hash + salt)	เลขบัตรประชาชน 13 หลัก + วันเกิด 8 หลัก (ววดดปปปป)
กุญแจเซ็น Token (HMAC)	`SESSION_SECRET` โดยตรง	`HMAC(SESSION_SECRET, "member-session-v1")` (Derived Key)
การคุ้มครอง Cookie	`HttpOnly`, `SameSite=Lax`, อายุ 7 วัน	`HttpOnly`, `SameSite=Lax`, อายุ 7 วัน
สิทธิ์การเข้าถึง	เข้าถึงหน้าจัดการสหกรณ์ทั้งหมด (ADMIN เข้าหน้า Users และ Audit Log ได้)	เข้าถึงได้เฉพาะแดชบอร์ดของตนเอง (`/member/*`)

> **ทำไมต้องใช้ Derived Secret สำหรับสมาชิก?**
>
> การนำ `SESSION_SECRET` มาสร้างกุญแจย่อยเฉพาะสมาชิก ทำให้แม้สมาชิกจะพยายามนำ Token ของตนเองไปเปลี่ยนชื่อ Cookie เป็น `session` เพื่อเข้าใช้งานในฐานะเจ้าหน้าที่ ระบบตรวจสอบของเจ้าหน้าที่จะถอดรหัสลายเซ็นดิจิทัลไม่ผ่านทันที ถือเป็นการป้องกันการยกระดับสิทธิ์ (Privilege Escalation) อย่างเด็ดขาด

5. ชั้นตรรกะข้อมูลและ API Endpoints (Data Layer)

ไฟล์ใน `server/lib/data/*` ทำหน้าที่เป็นคลังตรรกะข้อมูล (Service Layer) ที่ติดต่อกับฐานข้อมูลผ่าน Prisma และส่งผลลัพธ์กลับไปยัง API Routes:

โมดูล Data	ฟังก์ชันสำคัญ	ตรรกะการทำงาน

`members.js`	`getMembers()`
	`createMember()`
	`updateMember()`
	`setMemberStatus()`
		- ออกรหัสอัตโนมัติ `M-XXXX` (ต่อท้ายด้วยเลข 4 หลัก)
		- ตรวจสอบความถูกต้องของเลขบัตร 13 หลัก และเบอร์โทร 10 หลัก ป้องกันข้อมูลซ้ำ
		- แยก `photoUrl` ออกจากการดึงรายชื่อปกติเพื่อเพิ่มความเร็วในการโหลดตาราง

`employees.js`	`getEmployees()`
	`createEmployee()`
	`setEmployeeStatus()`
		- ออกรหัสอัตโนมัติ `E-XXXX`
		- โครงสร้างเหมือนสมาชิก แต่ไม่มีกระเป๋าเงินสะสม

`contracts.js`	`createContract()`
	`updateContractShares()`
	`getContractHistory()`
		- ตรวจสอบว่าทั้งสมาชิกและลูกจ้างมีสถานะ Active
		- ตรวจสอบว่าคู่นี้ยังไม่มีสัญญาเดิมที่เปิดใช้งานอยู่
		- ออกรหัสสัญญา `C-XXXX` และเก็บประวัติสัดส่วนย้อนหลัง

`purchases.js`	`lookupSeller()` `createPurchase()`	- ค้นหาผู้ขายรองรับทั้งรหัส M และ E (กรอกตัวเลขเดี่ยว ๆ เช่น `1` ได้) - คำนวณน้ำหนักยางแห้งสุทธิและยอดเงินรวม - ทำงานใน `prisma.$transaction` : บันทึกบิล `P-XXXX` พร้อมโอนยอดเงินเข้ากระเป๋าเงินสมาชิก (`walletBalance`) อัตโนมัติ
`withdrawals.js`	`lookupMember()` `createWithdrawal()`	- ตรวจสอบยอดเงินสะสมในกระเป๋าว่าเพียงพอกับยอดที่ต้องการเบิก - ทำงานใน `prisma.$transaction` : บันทึกบิลเบิก `W-XXXX` และตัดยอดเงินออกจากกระเป๋าเงินสมาชิกพร้อมกัน
`dividends.js`	`getDividendData()` `payDividend()`	- คำนวณผลรวมน้ำหนักยางแห้งสะสมทั้งปี พ.ศ. ของสมาชิกทุกคน - ป้องกันการจ่ายเงินปันผลซ้ำในปีเดียวกัน และห้ามจ่ายปันผลในปีอนาคต - สั่งจ่ายใน Transaction เดียว: สร้างประวัติ `D-XXXX` และบวกเงินเข้าทั้ง `walletBalance` และ `dividendBalance` ของสมาชิกทุกคน
`reference-price.js`	`getReferencePriceForDate()` `upsertReferencePrice()`	- บันทึกราคากลางประจำวัน (1 วันมีได้ 1 ราคา) - บันทึกประวัติการปรับราคาลงตาราง `ReferencePriceHistory` ทุกครั้ง
`member-portal.js`	`requireMemberPortal()` `getMemberFinanceHistory()`	- ตรวจสอบ Session สมาชิก หากถูกระงับจะบังคับ Logout ทันที - รวมประวัติการขาย, การเบิกเงิน, และเงินปันผลเป็น Timeline การเงินเดียว

6. ระบบหน้าบ้าน (HTML, CSS & DOM Engine)

ระบบหน้าบ้านได้รับการพัฒนาโดยไม่ใช้เฟรมเวิร์กขนาดใหญ่ แต่ใช้ **Native Web Standards** ที่เน้นความเร็ว ความเบา และความปลอดภัยสูงสุด:

6.1 การประกอบหน้าเว็บ (Server-Side HTML Includes)

ในไฟล์ `pages/*.html` มีการใช้คำสั่ง `<!-- @include partials/... -->` ซึ่งเซิร์ฟเวอร์ Express จะอ่านและนำชิ้นส่วน HTML มาประกอบก่อนส่งไปยังเบราว์เซอร์:

- `partials/head.html` : นำเข้าฟอนต์ Noto Sans Thai และ CSS สไตล์กลาง
- `partials/sidebar.html` : เมนูแถบข้างของฝั่งเจ้าหน้าที่ พร้อมแสดงจุดแจ้งเตือนรหัสผ่านเริ่มต้น
- `partials/member-sidebar.html` : เมนูแถบข้างและข้อมูลโปรไฟล์ของสมาชิก
- `partials/pagination.html` : ตัวแบ่งหน้าตาราง (Pagination Template)

6.2 กลไก Template & Data Slot (`public/js/core/dom.js`)

แทนที่จะใช้ `innerHTML` ซึ่งเสี่ยงต่อการถูกแฮ็กด้วย Cross-Site Scripting (XSS) ระบบเลือกใช้ HTML `<template>` ร่วมกับ `data-slot` :

```
// 1. คัดลอกแถวตารางจาก Template ใน HTML
const row = cloneTemplate("member-row-template");

// 2. แทรกข้อความอย่างปลอดภัยผ่าน textContent
setSlot(row, "memberCode", member.memberCode);
setSlot(row, "fullName", `${member.firstName} ${member.lastName}`);

// 3. ผูก Event Listener กับปุ่มที่ระบุ data-action
action(row, "edit").addEventListener("click", () => openEditModal(member));

tableBody.appendChild(row);
```

6.3 การจัดระเบียบ CSS Design System

CSS ถูกแยกออกเป็นสัดส่วนชัดเจนในโฟลเดอร์ `public/css/` :

- `base.css` : กำหนดค่าตัวแปรสีทั้งหมด (Tailwind v4 Semantic Tokens), Font, CSS Reset และกฎการพิมพ์ `@media print`
- `components.css` : คลาส UI กลาง เช่น ปุ่ม (`.btn`), อินพุต (`.input`), ตาราง (`.table`), ป็อปอัป (`.modal`), ป้ายสถานะ (`.badge`)
- `layout.css` : โครงร่างหน้าจอ 2 คอลัมน์ (Sidebar ซ้าย + Content ขวา)
- `receipt.css` : แบบฟอร์มใบเสร็จรับเงิน รองรับการสั่งพิมพ์จริงลงกระดาษ
- `member.css` : สไตล์เฉพาะของพอร์ทัลสมาชิก (การ์ดกระเป๋าเงินสีเขียว, กระดิ่งแจ้งเตือน)

7. ส่วนประกอบที่นำมาใช้ร่วมกัน (Shared Modules)

ชื่อโมดูล / ไฟล์	ประเภท	การนำไปใช้งานร่วมกัน
`shared/format.js`	Universal JS	ใช้ทั้ง **Client** และ **Server**: จัดรูปแบบเงินบาท (`฿1,234.50`), วันที่ภาษาไทย พ.ศ. (`7 ต.ค. 2569`), เวลาประเทศไทย (`Asia/Bangkok`), แปลงช่วงเดือน
`dom.js`	Core Client JS	ทุกหน้าเว็บใช้ฟังก์ชัน `byId()` , `cloneTemplate()` , `setSlot()` , `action()` , `setVisible()` ในการควบคุม UI
`combobox.js`	UI Component	ช่องเลือกข้อมูลอัจฉริยะ ใช้ในหน้ารับซื้อน้ำยาง, หน้าเบิกเงิน, หน้าสัญญาจ้าง, หน้าสรุปผลประกอบการ รองรับการพิมพ์ค้นหาและคีย์บอร์ด ARIA
`debounced-lookup.js`	Component Helper	ระบบค้นหาข้อมูลหน่วงเวลา 250ms พร้อม Token ป้องกันคำขอแข่งกัน ใช้ในหน้ารับซื้อ (ค้นหาผู้ขาย) และหน้าเบิกเงิน (ค้นหาสมาชิก)
`receipt.css`	Print Stylesheet	ใช้ร่วมกันในหน้ารับซื้อ, หน้าเบิกเงิน, หน้าสรุปผลประกอบการ และหน้าประวัติการขายของสมาชิก รองรับการพิมพ์จริงผ่านเครื่องพิมพ์
`status-dialogs.js`	UI Dialogs	ใช้ร่วมกันในหน้าสมาชิก, ลูกจ้าง, สัญญาจ้าง, และผู้ใช้งาน สำหรับกล่องยืนยันการระงับ/เปิดใช้งาน และลบข้อมูล

8. สูตรคำนวณและตรรกะทางธุรกิจที่สำคัญ

1. การคำนวณการรับซื้อน้ำยางสด (Purchase Calculation)

น้ำหนักเนื้อยางแห้งสุทธิ:

$$\text{dryWeightKg} = (\text{rawWeightKg} \times \text{dryPercentage}) \div 100$$

ยอดเงินรวมทั้งสิ้น:

$$\text{totalAmount} = \text{dryWeightKg} \times \text{marketPrice}$$

การจัดสรรเงินตามสัญญา MePair:
• **สมาชิกขายเอง:** เจ้าของสวนได้รับ `totalAmount` (100%) โอนเข้า `walletBalance`
• **ลูกจ้างส่งแทน:** ลูกจ้างได้รับสดทันที = `totalAmount × (employeeShare ÷ 100)`
เจ้าของสวนได้รับเข้ากระเป๋าเงินสะสม = `totalAmount - ส่วนของลูกจ้าง`

2. การเบิกเงินสะสมของสมาชิก (Withdrawal Calculation)

ยอดเงินคงเหลือในกระเป๋าหลังเบิก:

$$\text{balanceAfter} = \text{walletBalance} - \text{amount}$$

เงื่อนไขการเบิก: จำนวนเงินที่เบิกต้องมากกว่า 0 และต้องไม่เกินยอดสะสมคงเหลือ (`walletBalance`) โดยระบบจะตัดยอดเงินและบันทึกประวัติพร้อมกันใน Database Transaction

3. การคำนวณเงินปันผลประจำปี (Dividend Calculation)

ยอดเงินปันผลของสมาชิกแต่ละราย:

$$\text{amount} = \sum(\text{dryWeightKg ของสมาชิกในปี พ.ศ. นั้น}) \times \text{rate (บาท/กก.)}$$

ระบบล็อกเงินปันผล: เมื่อมีการกดจ่ายปันผลประจำปีแล้ว ระบบจะบันทึกลงตาราง `DividendPayment` และแสดงแบนเนอร์ล็อกหน้าจอคำนวณเงินปันผลของปีนั้นไว้ทันที จนกว่าปฏิทินจะเปลี่ยนเข้าสู่ปี พ.ศ. ถัดไปจริง ๆ จึงจะปลดล็อกให้คำนวณรอบใหม่
